

IDA Pro with its Functions and Features

Disassembler

- A reverse-engineering tool that **translates** compiled
 - machine code (binary zeros and ones) back into human-readable Assembly Language.
- While it doesn't recover the original C source code (which requires a decompiler), it allows you to see the exact instructions the CPU is executing.

Top Disassembler Tools

- **IDA Pro:**
 - The industry standard. Known for its powerful "Graph View," which visualizes code logic as flowcharts, and its extensive support for over 60 processor architectures.
- **Ghidra:**
 - A free, open-source suite developed by the NSA. It is highly popular because it includes both a high-quality disassembler and a built-in decompiler for free.
- **Binary Ninja:**
 - A modern, user-friendly alternative focused on automation and a clean API. It is often preferred by researchers for its "Intermediate Language" (IL) which simplifies complex assembly.
- **Radare2 / Cutter:**
 - A powerful, modular command-line framework (Radare2) with a dedicated graphical interface (Cutter). It is entirely open-source and highly scriptable.
- **x64dbg:**
 - Primarily a debugger for Windows, but it includes a robust real-time disassembler used heavily for malware analysis and binary patching.

Disassembler is a Static Analysis tool?

- They don't execute, but they do "simulate" or "emulate" small pieces of logic.
- Eg: they might look at a series of PUSH instructions and realize, "Oh, this is preparing to call the printf function," and label it for you.

- Integrated Debuggers:
 - Many disassemblers have a "**Debug Mode**" (like IDA's debugger or Ghidra's debugger).
 - When you click "Play," the tool switches roles—it launches the program and attaches to it.
 - At that moment, it is acting as a Debugger, not just a disassembler.

A Quick Example

- If you have a C line like
 - `int x = rand();`
 - Static (Disassembler):
 - You see `CALL rand`.
 - You know a random number will be generated, but you don't know what it is.
 - Dynamic (Debugger):
 - You run the line.
 - You look at the EAX register and see `0x5A3F`.
 - Now you know the exact number generated for that specific run.

- In professional reverse engineering, you usually use them together:
 - You use the **Disassembler** to find an interesting function (e.g., where a password is checked).
 - You set a "Breakpoint" and use a **Debugger** to pause the program at that exact line. Once paused, you look at the Registers to see the actual password sitting in memory.

Static vs. Dynamic Analysis

Feature	Disassembler (Static)	Debugger (Dynamic)
Execution	Code is not running.	Code is running in real-time.
Safety	Safe to use on malware/viruses.	Risk of infecting the host if not in a sandbox.
Visibility	You see the entire "map" of the code.	You only see the path the code is currently taking.
Data	You see hardcoded values and offsets.	You see actual live values in RAM and Registers.
Goal	To understand the structure and logic.	To understand the behavior and state.

Working of IDA Pro

- Uses a multi-stage process to transform raw binary data into human-readable assembly and then into C-like pseudocode.
- This process relies on **Recursive Descent Disassembly**
- Specialized intermediate representation called **Microcode**.

Recursive Descent Disassembly

- It is an algorithm that disassembles a binary by following its **Logical Control Flow**.
- Unlike simpler "Linear Sweep" methods that blindly translate every byte in a section, recursive descent "walks" through the code exactly like a CPU would.

How the Algorithm Works

- **Seed the Queue:**
 - The process begins at known entry points, such as the main function or exported symbols.
- **Follow the Flow:**
 - The **disassembler decodes an instruction and identifies where the execution goes next:**
 - Sequential Instructions (MOV, ADD):
 - It simply moves to the next immediate byte.
 - Unconditional Jumps (JMP):
 - It stops at the jump and starts disassembling at the target address.
 - Conditional Branches (JZ, BNE):
 - It creates two paths—it disassembles the "fall-through" path
 - If the condition is false, the CPU simply moves to the very next instruction in sequence.
 - &
 - adds the "jump target" to a queue for later processing.
 - Function Calls (CALL):
 - It queues the function's internal code for analysis
 - &
 - Continues disassembling the instruction immediately following the call.
- **Recursive Termination:**
 - The "recursion" stops when it hits a RET (Return) instruction
 - OR
 - a previously disassembled address, at which point it pulls the next pending address from its queue.

Drawback

- Recursive descent struggle with Indirect Jumps (e.g., `jmp EAX` or `call [edx+4]`).
- Since the target address is calculated at runtime, the disassembler cannot "statically" follow the path.
- To solve this, advanced tools like IDA Pro use **Heuristics** to recognize common patterns

Working Steps of IDA Pro

Step 1: Generating Assembly (IDA Pro)

- IDA Pro first acts as a disassembler to translate machine code into assembly.
- **Loading & Parsing:**
 - IDA **parses** the executable's header (e.g., ELF or PE) to **identify code sections (like .text) and data sections.**
- **Recursive Descent:**
 - Instead of just reading bytes in a straight line, IDA starts at the "entry point" and follows the code's logical flow.
 - **It follows jumps and function calls to find all reachable code.**
- **Instruction Decoding:**
 - It converts specific byte sequences into symbolic assembly instructions
 - E.g.: 0x89 becomes MOV based on the target processor architecture.

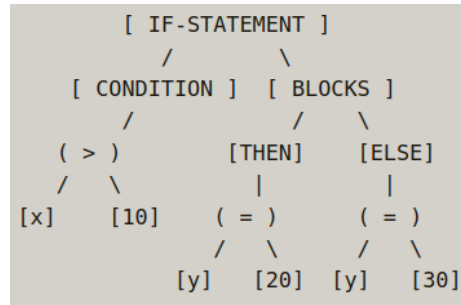
Step 2. Generating Intermediate Code (Hex-Rays)

- Once the assembly is mapped, the Hex-Rays Decompiler performs the heavy **lifting** of converting it to high-level logic.
- **Microcode (IR):**
 - The assembly is "**lifted**" into an architecture-independent **Intermediate Representation (IR)** called **Microcode**.
 - This simplifies complex CPU-specific instructions into universal primitive operations.
- **Optimization Levels:**
 - The microcode goes through several "maturity levels"
 - Ex: MMAT_GENERATED to MMAT_LVARS
 - During these stages, the decompiler:
 - **Removes "dead code"**
 - instructions that don't affect the final result.
 - Identifies function arguments and return values.
 - **Performs Global Optimization to simplify logic.**
- **CTree (Abstract Syntax Tree):**
 - From the final optimized microcode, Hex-Rays builds a CTree (C-Tree).
 - This is a tree structure where nodes represent C constructs like if statements, for loops, and function calls.

Abstract Syntax Tree

- In the context of decompilation, an Abstract Syntax Tree (AST) is the final bridge between "math-like" assembly and "structured" C code.
- It represents the **logical hierarchy of a program** without caring about semicolons or curly braces
- The decompiler only cares about the **relationship between the data**.
- It knows that the > node must evaluate to true or false, and that choice determines which branch of the IF-STATEMENT node to follow.

```
if (x > 10) {  
    y = 20;  
} else {  
    y = 30;  
}
```



How Hex-Rays (F5) uses AST?

1. Assembly:

You have CMP and JBE (Jump if Below or Equal).

2. Lifting:

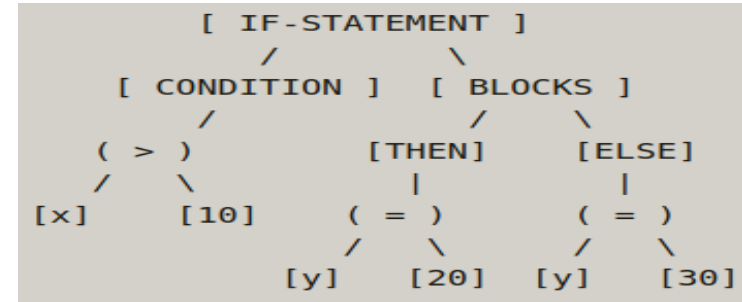
The instructions are turned into Microcode (e.g., if x <= 10 goto LABEL_ELSE).

3. AST Generation:

The decompiler realizes this is a high-level "If-Else" pattern and builds the tree shown below.

4. C-Output: The decompiler "walks" the tree:

- It hits the IF node: Writes if (
 - It hits the CONDITION node: Writes x > 10
 - It hits the THEN node: Writes) { y = 20; }
 - ...and so on.
-
- In IDA Pro's SDK, this AST is specifically called the **citem_t** (C-Item).
 - Every line you see in the decompiler window is just a text representation of these underlying tree nodes.



3. Final Transformation to Pseudocode

- Finally, the decompiler walks the CTree to generate the final text output you see when you press F5.
- **Type Inference:**
 - It **attempts to guess variable types** (e.g., int, char*) based on how they are used in the code.
- **Interactivity:**
 - Because it's an "Interactive Disassembler," you can **manually rename variables or change** types, and the decompiler will **re-run its analysis** to propagate those changes throughout the pseudocode.

Fast Library Identification and Recognition Technology (FLIRT)

- A signature-matching technology used by IDA Pro to
 - automatically identify and
 - name standard library functions (like printf, malloc, or strcpy) in compiled binaries.
- It is essential for analyzing **stripped binaries**—files where the developer has removed all function names to save space or hide logic
- Allows you to skip the tedious work of reverse engineering code that has already been written by someone else.

How FLIRT Works?

- The technology identifies code by matching byte patterns at the start of functions against a database of known signatures.
 - **Pattern Creation:**
 - A signature is created from the first 32 bytes of a function.
 - **Wildcarding**
 - Since addresses and certain registers can change depending on how a program is linked, FLIRT uses wildcards to **skip** these "variant" bytes, matching only the **stable opcode patterns**.
 - Like memory addresses or offsets that change between different builds
 - **Recursive Matching:**
 - To distinguish between functions that start with **identical bytes**, FLIRT can look at the functions they call (references) to resolve the correct identity.
 - **Database Integration:**
 - IDA comes with pre-built signatures for major compilers (like GCC or Visual Studio) and common libraries (like OpenSSL).
- **Identification:**
 - If the pattern matches a known library function, IDA automatically renames the function (e.g., from sub_4010A0 to printf).

The FLAIR Toolset

- While IDA provides many signatures, you can create your own using the **FLAIR** (Fast Library Acquisition for Identification and Recognition) toolkit:
 - **pelf / plb:**
 - Parsers that extract patterns from static library files into a textual **.pat** format.
 - **sigmake:**
 - A utility that converts **.pat** files into a compressed **.sig** file that IDA can load.
 - **.exc files:**
 - If two different functions have identical patterns, **sigmake** creates an "exclusion" file, requiring you to **manually resolve the collision**.

Identifying Malware Capabilities

- By identifying standard library functions, you gain immediate context.
- If FLIRT identifies functions from:
 - OpenSSL:
 - The malware likely uses encryption.
 - Libcurl/Socket libraries:
 - The malware has networking capabilities.
 - Compression libraries (zlib):
 - The malware might be decompressing a second-stage payload.

Workflow Acceleration

- Once FLIRT renames library functions, IDA's **Graph View** and **Decompiler output** become significantly more readable.
- Instead of seeing a call to an unknown address, you see:
 - `if (strcmp(input, "malicious_command") == 0) { ... }`

Limitations & Evasion

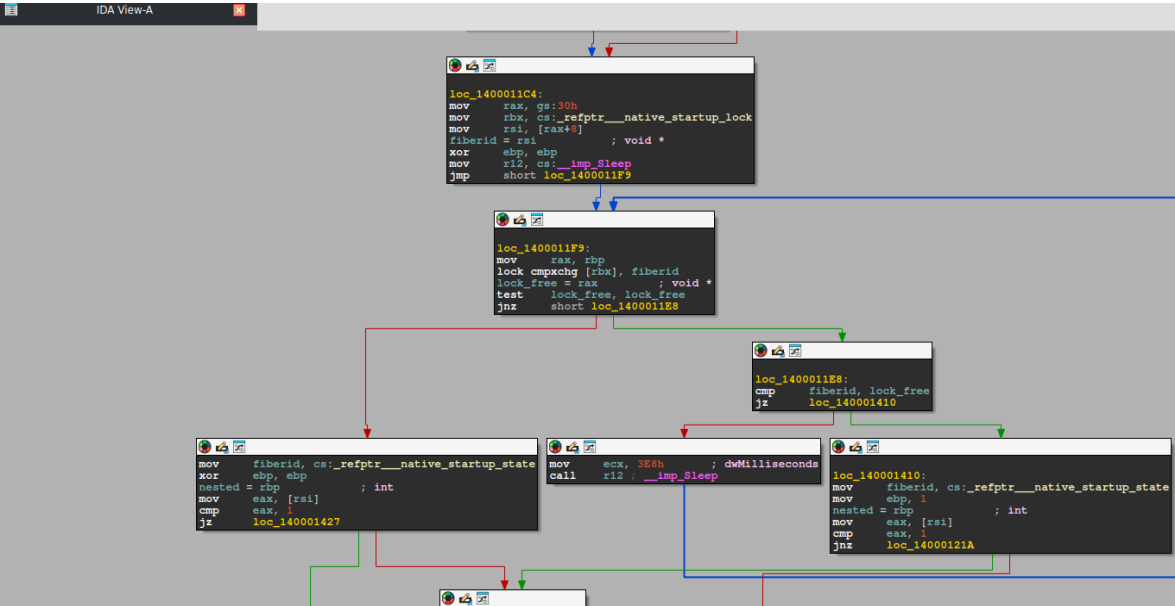
- **Evasion:**
 - Advanced malware authors may slightly modify library functions or use custom compilers to break FLIRT patterns.
- **Collisions:**
 - Sometimes different functions have identical patterns (especially very short ones), which can lead to misidentification.
- **Architecture Specificity:**
 - Signatures are specific to the architecture (x86, x64, ARM) and the OS.

The IDA Pro Interface

- Disassembly Window Modes
 - Graph (the default)
 - Text

To switch between modes, press the spacebar

IDA View (Graph Mode)



- **Green Arrows:**
 - The conditional jump is taken (the condition is true).
- **Red Arrows:**
 - The conditional jump is not taken (the condition is false, fall-through).
- **Blue Arrows:**
 - Unconditional jumps or normal flow.

The arrow direction shows the program's flow;

- **upward arrows typically denote a loop situation.**

Graph Mode

- IDA Pro excludes certain recommended information to be displayed, such as line numbers and operation codes.
- To change these options, select Options  General, and then select Line prefixes and set the Number of Opcode Bytes to 6.
- Because most instructions contain 6 or fewer bytes,
 - this setting will allow you to see the memory locations and opcode values for each instruction in the code listing.
 - If these settings make everything scroll off the screen to the right, try setting the Instruction Indentation to 8.

Text Mode

- Use it to view data regions of a binary
- The left portion of the text-mode display is known as the arrows window and shows the program's nonlinear flow.
- Solid lines mark unconditional jumps, and dashed lines mark conditional jumps.
- Arrows facing up indicate a loop.

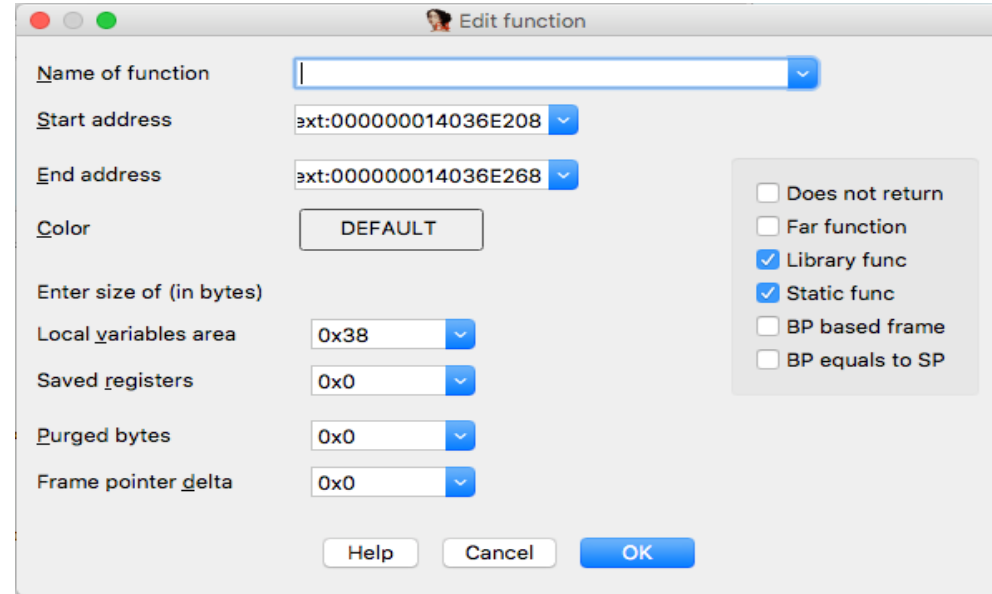
Text Mode

```
.text:00401040 sub_401040 proc near ; CODE XREF: sub_401040+2A1p
.text:00401040 var_18 = dword ptr -18h
.text:00401040 var_14 = dword ptr -14h
.text:00401040 var_10 = dword ptr -10h
.text:00401040 var_C = dword ptr -0Ch
.text:00401040 var_8 = dword ptr -8
.text:00401040 var_4 = dword ptr -4
.text:00401040
1 .text:00401040 55 push ebp
.text:00401041 89 ES mov ebp, esp
.text:00401043 83 EC 18 sub esp, 18h
.text:00401046 C7 45 F4 00 00 00+ mov [ebp+var_C], 0
.text:0040104D C7 45 F0 00 00 00+ mov [ebp+var_10], 0
.text:00401054 C7 45 F0 64 00 00+ mov [ebp+var_4], 64h
.text:00401058
.text:00401058 loc_401058: ; CODE XREF: sub_401040+5C1j
.text:00401058 03 7D FC 01 cmp [ebp+var_4], 1
.text:0040105F 7E 3D jle short locret_40109E
.text:00401061 C7 45 F0 00 00 00+ mov [ebp+var_10], 0
.text:00401068 8B 45 F8 mov eax, [ebp+var_8]
.text:0040106B 8B 45 FC add eax, [ebp+var_4]
.text:0040106E 8B 45 F4 mov [ebp+var_C], eax
.text:00401071 83 7D F4 1E cmp [ebp+var_C], 1Eh
.text:00401075 75 07 jnz short loc_40107E
.text:00401077 C7 45 F0 01 00 00+ mov [ebp+var_10], 1
.text:0040107E
.text:0040107E loc_40107E: ; CODE XREF: sub_401040+5T1j
3 .text:0040107E 03 7D F4 00 cmp [ebp+var_C], 0
.text:00401082 75 13 jnz short loc_401097
.text:00401084 8B 45 FC mov eax, [ebp+var_4]
.text:00401087 8B 44 24 04 mov [esp+18h+var_14], eax
.text:0040108B C7 04 24 20 20 40+ mov [esp+18h+var_10], offset aPrintNumberB ; "Print Number= %d\n"
.text:00401092 E8 01 00 00 00 call printf
.text:00401097
.text:00401097 loc_401097: ; CODE XREF: sub_401040+421j
.text:00401097 8D 45 FC lea eax, [ebp+var_4]
.text:0040109A FF 08 dec dword ptr [eax]
.text:0040109C EB 8D jmp short loc_401058
.text:0040109E
.text:0040109E locret_40109E: ; CODE XREF: sub_401040+1F1j
.text:0040109E C9 leave
.text:0040109F C3 retn
.text:0040109F sub_401040 endp
```

- Displays the memory address (0040105B) and section name (.text) in which the opcodes (83EC18) will reside in memory 1.
- The left portion of the text-mode display is known as the arrows window and shows the program's nonlinear flow.
- Solid lines mark unconditional jumps, and dashed lines mark conditional jumps. Arrows facing up indicate a loop.
- The example includes the stack layout for the function at 2 and a comment (beginning with a semicolon) that was automatically added by IDA Pro 3.

Functions window

- Every column value can be either a dot or the same column character. Those columns are boolean and the column character stands for "True" while the dot stands for "False".
 - R stands for "Returns" and is True (has an "R" in the column's value instead of a dot) if the function returns. Functions may not return and this requires special handling by IDA and the reverse engineer.
 - F stands for "Far" and is true for functions that are far functions, or false otherwise. Indicates calls across memory segments.
 - L stands for "Library" function, and is true for functions IDA recognised as library functions (usually using FLIRT).
 - S stands for "Static" function.
 - B stands for a E/R/BP stack frame based function. As part of IDA's analysis functions that use the xBP register for easier stack access are marked so.
 - T stands for Type and indicates whether a function has type information assigned. Either manually or automatically by IDA using FLIRT.
 - the equal sign (=) indicates the frame pointer is equal to the initial stack pointer.



Edit function

Name of function:

Start address:

End address:

Color:

Enter size of (in bytes):

Local variables area:

Saved registers:

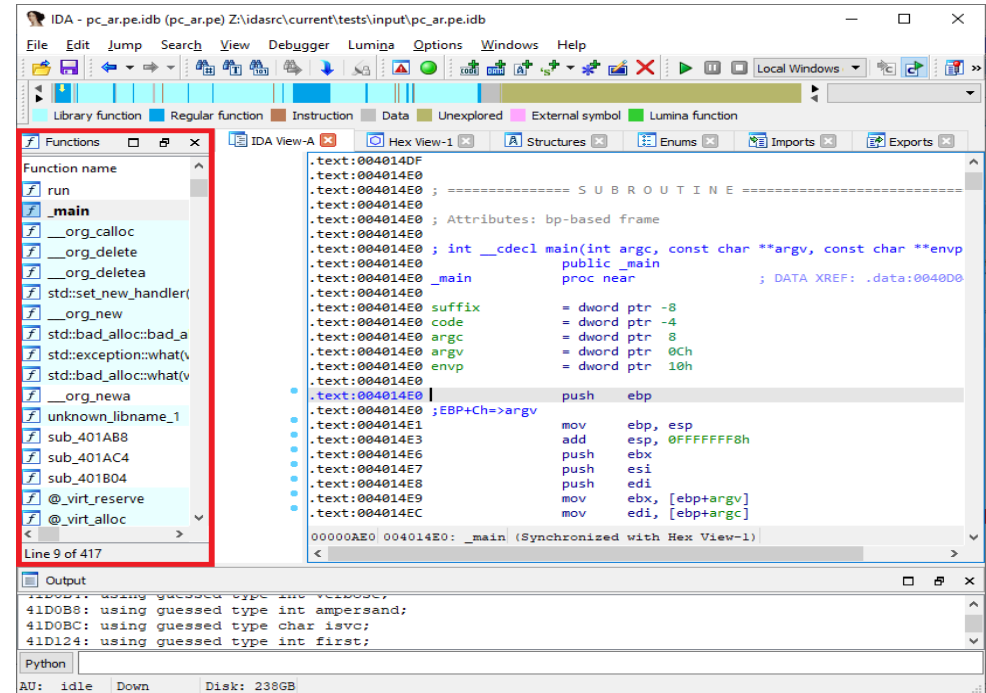
Purged bytes:

Frame pointer delta:

☐ Does not return
☐ Far function
☒ Library func
☒ Static func
☐ BP based frame
☐ BP equals to SP

Names Window

- Displays a list of all named addresses in the program. Each entry represents a memory address that IDA has assigned a symbolic name to
- These names may be automatically generated by IDA or manually assigned by the analyst



Names Window

- Functions
 - Named procedures detected by IDA.
 - Example: sub_401000, main, printf.
- Named Code Locations
 - Specific labeled instruction addresses used for easier navigation.
- Named Data
 - Variables or memory locations containing structured data.
 - Example: global_var, buffer.
- Strings
 - ASCII or Unicode strings discovered in the binary.
 - Example: "Error occurred", "Enter password".

 Library function  Regular function  Instruction  Data  Unexplored  External symbol  Lumina function

IDA PRO NAMES WINDOW: COLOR-CODE KEY

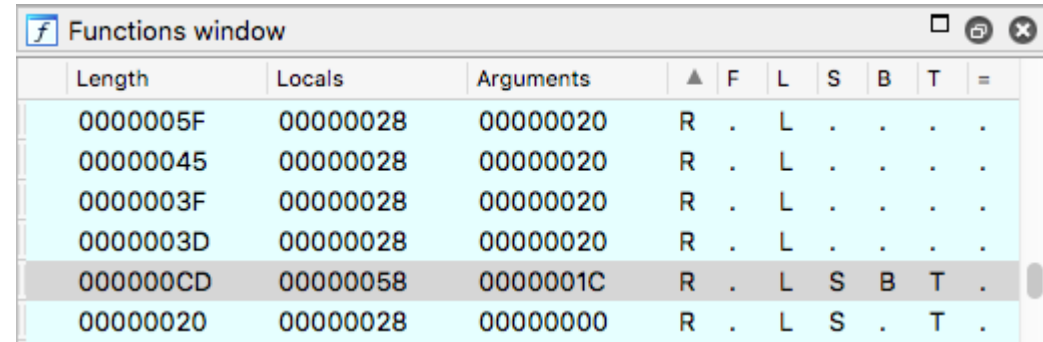
Color	Meaning	Description
	Function	 Represents custom user-defined logic or auto-analyzed program code.
	Library Function	 Indicates standard compiler functions (e.g., `printf`, `strlen`), identified via FLIRT.
	Imported Name	 External APIs called from other modules (DLLs, libraries) like `MessageBoxA`.
	Data	 Global variables, constants, buffers, or named locations in the data segments.
	String	 Named text sequences (ASCII, Unicode) often containing credentials or messages.
	Code Label	 A named address pointing to an instruction within a function (jump target or loop start).

- Strings window:
 - Shows all strings. By default, this list shows only ASCII strings longer than five characters. You can change this by right-clicking in the Strings window and selecting Setup.Imports window. Lists all imports for a file.
- Exports window.
 - Lists all the exported functions for a file.
 - This window is useful when you're analyzing DLLs.
- Structures window
 - Lists the layout of all active data structures.
 - The window also provides you the ability to create your own data structures for use as memory layout templates.

Useful Windows for Analysis

- Functions window

- Lists all functions in the executable and shows the length of each. You can sort by function length and filter for large, complicated functions that are likely to be interesting, while excluding tiny functions in the process.
- This window also associates flags with each function (F, L, S, and so on)
- The most useful of which, L, indicates library functions.
- The L flag can save you time during analysis, because you can identify and skip these compiler-generated functions.



Length	Locals	Arguments	▲	F	L	S	B	T	=
0000005F	00000028	00000020	R	.	L	.	.	.	.
00000045	00000028	00000020	R	.	L	.	.	.	.
0000003F	00000028	00000020	R	.	L	.	.	.	.
0000003D	00000028	00000020	R	.	L	.	.	.	.
000000CD	00000058	0000001C	R	.	L	S	B	T	.
00000020	00000028	00000000	R	.	L	S	.	T	.

Navigating IDA Pro

- Many windows are linked to the disassembly window.
- Double-clicking an entry within the Imports window or Strings window will take you directly to that entry.

Using Links and Cross-References

```
00401075      jnz      short 1 loc_40107E
00401077      mov     [ebp+var_10], 1
0040107E loc_40107E:      ; CODE XREF: 1 2 sub_401040+35j

0040107E      cmp     [ebp+var_C], 0
00401082      jnz     short 1 loc_401097
00401084      mov     eax, [ebp+var_4]
00401087      mov     [esp+18h+var_14], eax
0040108B      mov     [esp+18h+var_18], offset 1 aPrintNumberD ; "Print Number=
%d\n"
00401092      call    1printf
00401097      call    1sub_4010A0
```

- The following are the most common types of links:
 - **Sub links**
 - links to the start of functions such as printf and sub_4010A0.
 - **Loc links**
 - links to jump destinations such as loc_40107E and loc_401097.
 - **Offset links**
 - links to an offset in memory.

Using Links and Cross-References

```
00401075      jnz      short 1 loc_40107E
00401077      mov     [ebp+var_10], 1
0040107E loc_40107E:      ; CODE XREF: 1 2 sub_401040+35j

0040107E      cmp     [ebp+var_C], 0
00401082      jnz     short 1 loc_401097
00401084      mov     eax, [ebp+var_4]
00401087      mov     [esp+18h+var_14], eax
0040108B      mov     [esp+18h+var_18], offset 1 aPrintNumberD ; "Print Number=
%d\n"
00401092      call    1printf
00401097      call    1sub_4010A0
```

- Cross-references (shown at 2 in the listing) are useful for jumping the display to the referencing location: 0x401075 in this example.
- Because strings are typically references, they are also navigational links. For example, aPrintNumberD can be used to jump the display to where that string is defined in memory.
- CODE XREF: sub_401040+35j is an annotation generated by IDA Pro to help you understand the program's control flow.

Using Cross-References

- Code cross-reference at 1 that tells us that this function (sub_401000) is called from inside the main function at offset 0x3 into the main function.
- The code cross-reference for the jump at 2 tells us which jump takes us to this location, which in this example corresponds to the location marked at 3.
- We know this because at offset 0x19 into sub_401000 is the jmp at memory address 0x401019.

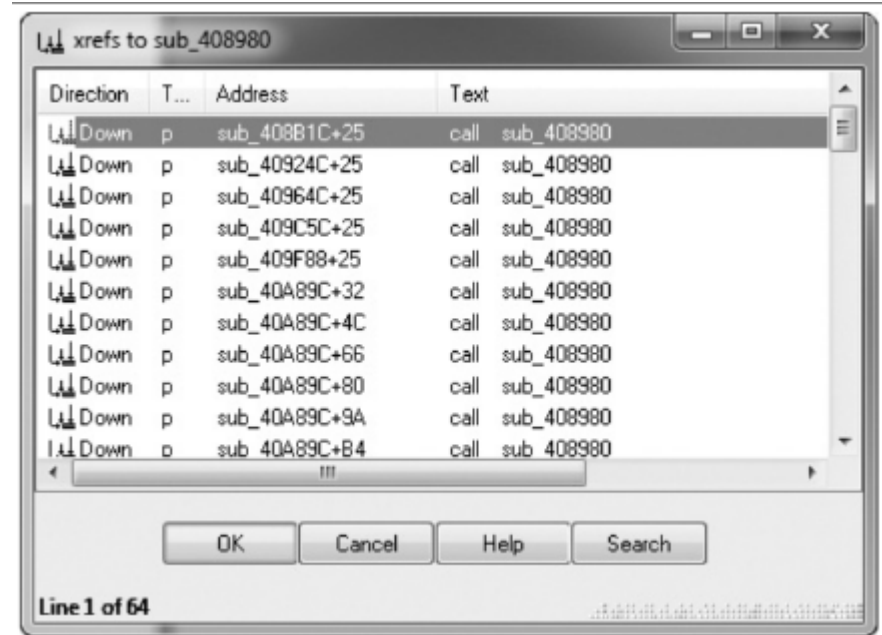
```
00401000      sub_401000      proc near      ; 1 CODE XREF: main+3p
00401000      push     ebp
00401001      mov      ebp, esp
00401003 loc401003:      ; 2 CODE XREF: sub_401000+19j
00401003      mov      eax, 1
00401008      test     eax, eax
0040100A      jz       short loc_40101B
0040100C      push     offset aLoop      ; "Loop\n"
00401011      call     printf
00401016      add      esp, 4
00401019      jmp      short loc_401003 3
```

Using Cross-References

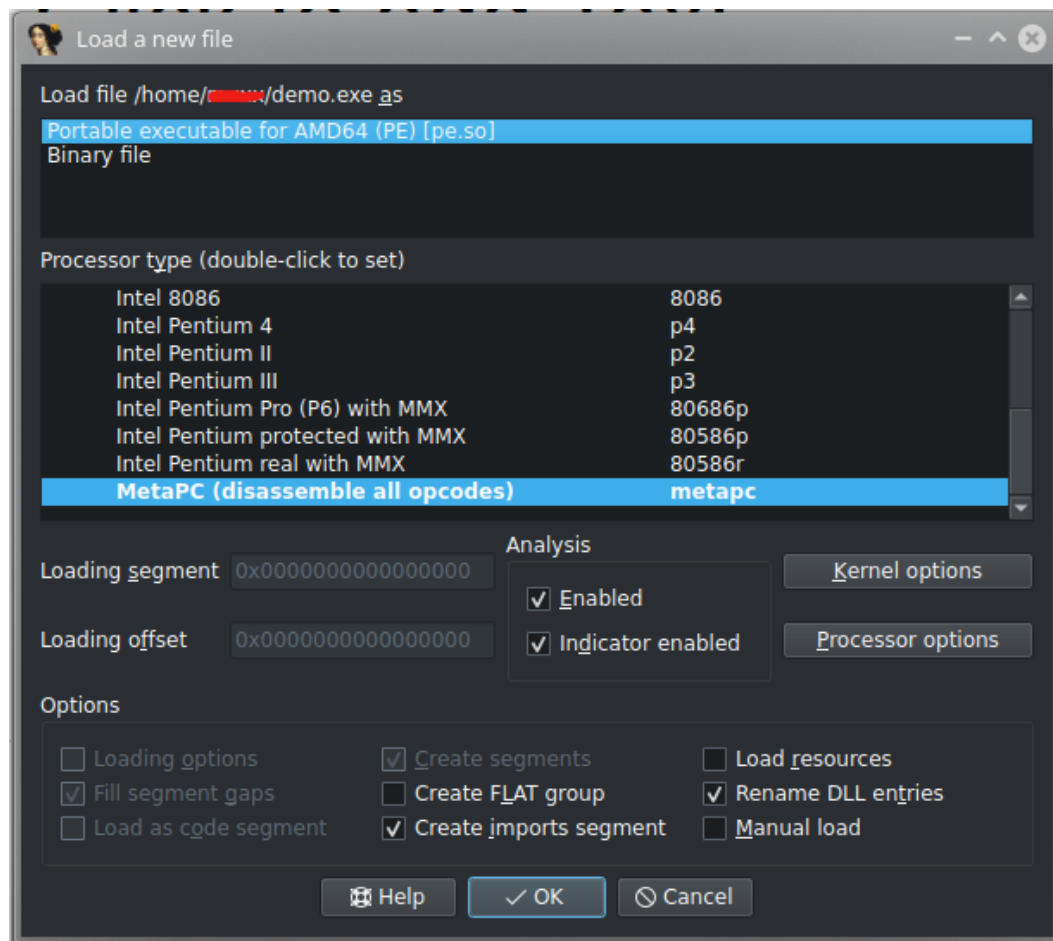
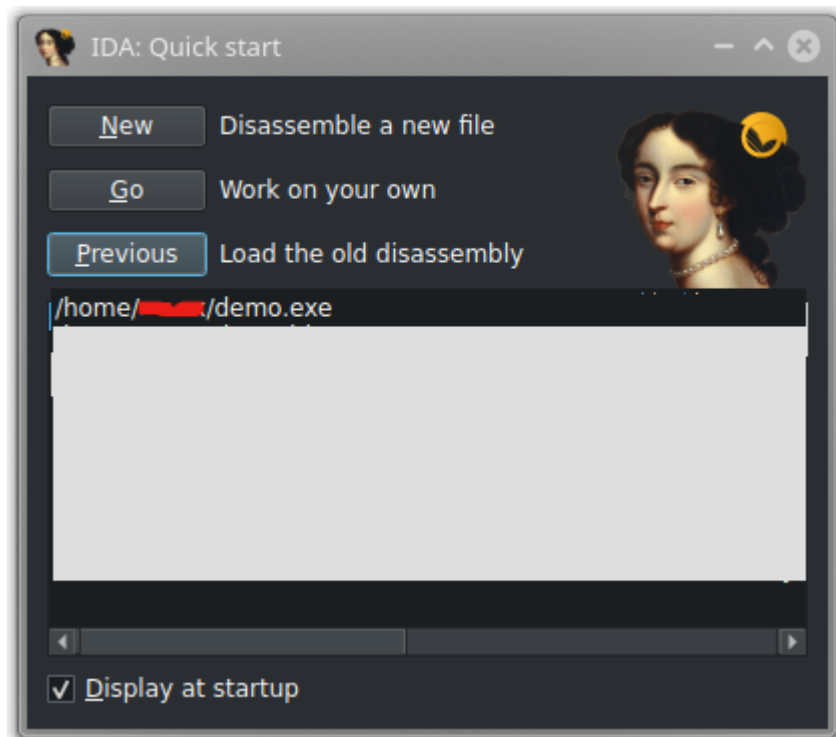
- By default, IDA Pro shows only a couple of cross-references for any given function, even though many may occur when a function is called.
- To view all the cross-references for a function, click the function name and press X on your keyboard.
- The window that pops up should list all locations where this function is called.

Xrefs window

- shows a list of cross-references for sub_408980, you can see that this function is called 64 times
- Double-click any entry in the Xrefs window to go to the corresponding reference in the disassembly window.



IDA Pro



Hex View

```
0000000140001150 00 00 48 8D 15 C7 5E 00 00 48 8D 0D C8 5E 00 00 ..H.....H.....
0000000140001160 8B 00 89 05 9C 5E 00 00 48 8D 05 95 5E 00 00 48 .....^..H.....^..H
0000000140001170 89 44 24 20 48 8B 05 95 32 00 00 44 8B 08 E8 95 ..D$.H...2...D....
0000000140001180 15 00 00 90 48 83 C4 38 C3 0F 1F 80 00 00 00 00 ...H.....
0000000140001190 41 55 41 54 55 57 56 53 48 81 EC 98 00 00 00 B9 AUATUWVSH.....
00000001400011A0 0D 00 00 00 31 C0 4C 8D 44 24 20 4C 89 C7 F3 48 .....1...D$.L...H
00000001400011B0 AB 48 8B 3D A8 32 00 00 44 8B 0F 45 85 C9 0F 85 ..H.=2...D...E....
00000001400011C0 AC 02 00 00 65 48 8B 04 25 30 00 00 00 48 8B 1D .....eH...%0...H..
00000001400011D0 CC 31 00 00 48 8B 70 08 31 ED 4C 8B 25 CB 6F 00 .....H.p.1...%....
00000001400011E0 00 EB 16 0F 1F 44 00 00 48 39 C6 0F 84 1F 02 00 .....D...H9.....
00000001400011F0 00 B9 E8 03 00 00 41 FF D4 48 89 E8 F0 48 0F B1 .....A.....
0000000140001200 33 48 85 C0 75 E2 48 8B 35 A3 31 00 00 31 ED 8B 3H.....5.1...1..
0000000140001210 06 83 F8 01 0F 84 0D 02 00 00 8B 06 85 C0 0F 84 .....
0000000140001220 7C 02 00 00 C7 05 DA 5D 00 00 01 00 00 00 8B 06 |.....
0000000140001230 83 F8 01 0F 84 03 02 00 00 85 ED 0F 84 1C 02 00 .....
0000000140001240 00 48 8B 05 E8 30 00 00 48 8B 00 48 85 C0 74 0C ..H.....H...H....
0000000140001250 45 31 C0 BA 02 00 00 00 31 C9 FF D0 E8 9F 08 00 E1.....1.....
0000000140001260 00 48 8D 0D E8 0B 00 00 FF 15 36 6F 00 00 48 8B ..H.....60...H..
0000000140001270 15 1B 31 00 00 48 8D 0D 84 FD FF FF 48 89 02 E8 ...1..H.....H....
0000000140001280 DC 14 00 00 E8 D7 05 00 00 48 8B 05 B0 30 00 00 .....H.....0...
0000000140001290 48 89 05 F1 5E 00 00 E8 D4 14 00 00 31 C9 48 8B H.....1...
00000001400012A0 00 48 85 C0 75 1C EB 58 0F 1F 84 00 00 00 00 00 ..H.....
00000001400012B0 84 D2 74 45 83 E1 01 74 27 B9 01 00 00 00 48 83 ...E.....'.....H.
00000001400012C0 C0 01 0F B6 10 80 FA 20 7E E6 41 89 C8 41 83 F0 .....~.....
00000001400012D0 01 80 FA 22 41 0F 44 C8 EB E4 66 0F 1F 44 00 00 ..."A.D.....D...
00000001400012E0 84 D2 74 15 0F 1F 40 00 0F B6 50 01 48 83 C0 01 .....@...P.H....
00000001400012F0 84 D2 74 05 80 FA 20 7E EF 48 89 05 80 5E 00 00 .....~.....^...
0000000140001300 44 8B 07 45 85 C0 74 16 B8 0A 00 00 00 F6 44 24 D...E.....
0000000140001310 5C 01 0F 85 E8 00 00 00 89 05 E2 1C 00 00 8B 1D \.....
0000000140001320 04 5D 00 00 44 8D 63 01 4D 63 E4 49 C1 E4 03 4C .].D.c.Me.....
0000000140001330 89 E1 E8 71 13 00 00 4C 8B 2D E2 5C 00 00 48 89 .....L...-....H..
0000000140001340 C7 85 DB 7E 46 8D 6B FF 31 DB 66 0F 1F 44 00 00 ..F.k.1...D....
0000000140001350 49 8B 4C DD 00 E8 36 13 00 00 48 8D 70 01 48 89 I.L.....H.p.H..
0000000140001360 F1 E8 42 13 00 00 49 89 F0 48 89 04 DF 49 8B 54 .....I.....T...
0000000140001370 DD 00 48 89 C1 E8 26 13 00 00 48 89 D8 48 83 C3 ..H...&...H....
0000000140001380 01 48 39 C5 75 CA 4A 8D 44 27 F8 48 C7 00 00 00 .H9....D'.H....
0000000140001390 00 00 48 89 3D 87 5C 00 00 E8 B2 02 00 00 48 8B ..H.=\.....H...
00000001400013A0 05 AB 2F 00 00 4C 8B 05 6C 5C 00 00 8B 0D 76 5C ../.L..1\...v\
00000001400013B0 00 00 48 8B 00 4C 89 00 48 8B 15 61 5C 00 00 E8 ..H...L...H...a\
00000001400013C0 B1 01 00 00 8B 0D 42 5C 00 00 89 05 40 5C 00 00 .....B\....@\\..
00000001400013D0 85 C9 0F 84 E6 00 00 00 8B 15 2A 5C 00 00 85 D2 .....*\.....
00000001400013E0 0F 84 9A 00 00 00 48 81 C4 98 00 00 00 5B 5E 5F .....H.E....[^_
00000001400013F0 5D 41 5C 41 5D C3 66 2E 0F 1F 84 00 00 00 00 00 ]A\A].....
0000000140001400 0F B7 44 24 60 E9 0E FF FF FF 66 0F 1F 44 00 00 ..D$.f...f.D...
0000000140001410 48 8B 35 99 2F 00 00 BD 01 00 00 00 8B 06 83 F8 H.5./.....
0000000140001420 01 0F 85 F3 FD FF FF B9 1F 00 00 00 E8 C7 12 00 .....
0000000140001430 00 8B 06 83 F8 01 0F 85 FD FD FF FF 48 8B 15 8D .....H....
0000000140001440 2F 00 00 48 8B 0D 76 2F 00 00 E8 99 12 00 00 C7 /..H...v/.....
0000000140001450 06 02 00 00 00 85 ED 0F 85 E4 FD FF FF 31 C0 48 .....1...
0000000140001460 87 03 E9 DA FD FF FF 66 0F 1F 84 00 00 00 00 00 .....f....
0000000140001470 4C 89 C1 FF 15 13 6D 00 00 E9 46 FD FF FF 66 90 L....m.....f..
00000790 0000000140001190: __tmainCRTStartup (Synchronized with IDA View-A)
```